

インタラクティブメディア 最終課題 レポート

学籍番号：2331033

氏名：桑島 大知

2026 年 1 月 31 日

1 作品について

今回、インタラクティブメディアの最終課題として、常に前進する魚を操作して対戦するゲームを作成した。作品は完成せず、魚を 2 匹操作して遊ぶことができる状態までである。チンアナゴを避けながら、相手の魚よりチンアナゴに当たらずに残っていると勝利するというルールである。

2 工夫した点

第一講時の最終課題説明のように線画で作品を作ることに注力した。

オブジェクト指向で記述し、各クラスの役割を明確にした。また、魚を 2 匹操作できるようにし、対戦できるようにした。これによりプログラムの管理をしやすくなり、拡張性を高めることで完全な感性ではなくとも遊べる状態まで持っていくことができ、アジャイルの開発を実現し当初の構想まで持っていくことができなくとも問題なく動くようにできた。また、再利用性を高めることができ、文字列などの表示に関するクラスを汎用的に作成できた。

回転は四元数を用いて実装した。一方で透視投影変換、カメラ座標変換は同時座標系の行列で実装することになった。

魚の加速機能で単純にの速度を定数倍するのではなく、上限をもうけ加減速度的に変化させるようにしたことで、より自然な動きになった。また、線画では人間の認知的に上下前後関係が分かりづらいため、魚の影として体の軸と曲がり方を床面に投影して描画することで、魚の位置関係が分かりやすくなるように工夫した。全体的に線画で 3D を扱うためのインタラクションを意識した作品になった。

3 苦労した点

多くのことを妥協することになった。まず魚の動きであるが、魚特有の動きを実装することができなかった。本来は数学的に正しい魚の体の動かし方を調査し実装することが目的の一つであったが、時間が足りず、単純にキーボード入力に応じて魚の向きを変え、直進するだけの動きになってしまった。ただし、視覚的に魚がどううごいているのかを分かりやすくするために、魚の体を構成する頂点をサイン波で変位させることで、魚が泳いでいるように見えるようにした。また、魚を 2 匹操作できるようにしたが、入力キーを変えるためにメソッドを複製する必要があるため、冗長なコードになってしまった。そして、チンアナゴも描画する際に円柱で

表現する簡易的な表現にとどまったが、視覚的に動いているのが分かりやすいように x-z 平面に描いた円を毎フレーム再計算して線形に移動させて描画した。

今まで作成したコードのリファクタリングに時間がかかり、追加機能の実装に時間をたくさん割くことができなかった。

4 実装

4.1 操作方法

1P は WASD キーで上下左右を操作し、スペースキーで加速するようにした。2P は矢印キーで上下左右を操作し、右コントロールキーで加速するようにした。

4.2 クラス構成

- GameManager: ゲーム全体を管理するクラス。各オブジェクトの生成、更新、描画を行う。
- Fish: 魚を表現するクラス。位置、速度、向きの管理と更新、描画を行う。
- ChineseEel: チンアナゴを表現するクラス。位置、速度の管理と更新、描画を行う。
- Frame: 水槽の枠を表現するクラス。位置の管理と描画を行う。
- Camera: カメラを表現するクラス。視点の管理と更新を行う。
- StringsBlock: 文字列表示を行うクラス。位置、フォントサイズ、色の管理と描画を行う。

また、補助関数のファイルとして、四元数の計算を行う複数の関数をもつ `quarternions.py`、簡易 viewport clipping 計算する関数、x-z 平面に円を描く関数を持つ `object_module.py`、ウィンドウ管理を行い実行する `main` 関数をもつ実行部分である `main.py` の 4 つを実装した。

4.3 コード

まず、実行部分である `main.py` のコードを示す。このファイルの 77 行目以降が実行部分である。左上を原点とし x 座標が右方向に、y 座標が下方向に増加するものであった座標系を中心原点の x 軸が右方向に、y 軸が上方向に増加する座標系に変換している課題用に配布されたコードを流用している。

Listing 1 main.py

```
1 import sys
2 import pygame
3 import numpy as np
4 from Camera import *
5 from Frame import *
6 from Fish import *
7 from StringsBlock import *
8 from ChineseEel import *
9 from GameManager import GameManager
10 #魚を泳がせてチンアナゴをよけるゲーム
11 #現状はチンアナゴに触れるとゲームオーバーってだけです、
12 #得点システムとか、餌を食べて糞をすとか、対戦要素とかを入れるつもりでした。
13 #操作方法は W上、:S下、:A左、:D右、:Rリセット、:ESC終了です。:
14 #スペースキーで加速ができますが、餌システムが未実装なのであまり意味はありません。
15 #魚の向きは移動方向に合わせて自動で変わります。
```

```

16 #また、今回はまだ壁、床に当たっても大丈夫なようにしてある。
17
18
19
20
21 # 定数の設定
22 WIDTH,HEIGHT=1000, 1000 # 画面の幅と高さ
23 scrcentr=np.array([(WIDTH-1)/2, (HEIGHT-1)/2]) # 画面の中心の座標
24 scale = (WIDTH-1)/2 # 画面サイズの座標に変換する際の拡大率
25 fps = 60 # フレーム秒/
26 dt = 1/fps # 秒フレーム/
27
28 def main():
29     # の初期化 Pygame
30     pygame.init()
31
32     game_manager = GameManager()
33     # 画面の生成
34     screen = pygame.display.set_mode((WIDTH, HEIGHT))
35     # 描画バッファの作成
36     buf = pygame.Surface((WIDTH, HEIGHT))
37
38     # タイトルバーの設定 (表示する文字を指定)
39     pygame.display.set_caption("Fish Simulation")
40
41     clock = pygame.time.Clock() # 時計作成
42
43     Key_vel = 1e-2
44
45     running = True # ループ処理の実行を継続するフラグ
46
47     while running:
48
49         buf.fill((0, 0, 0)) # 描画バッファを背景色 (RGB)で塗りつぶす
50
51         game_manager.update(key_input=pygame.key.get_pressed(), key_vel=Key_vel, buf=buf,
52                             scrcentr=scrcentr, scale=scale)
53
54         # 画像座標の左下を原点にするための上下反転処理
55         flippedbuf = pygame.transform.flip(buf, 0, 1) # 画像全体の上下反転
56         screen.blit(flippedbuf, (0,0)) # 反転した画像を画面へ転送
57
58         pygame.display.update() # 表示更新
59         delta = clock.tick(fps) # 制限 fps
60         # print(delta, "ms")
61
62         # イベント処理
63         for event in pygame.event.get():
64             # 閉じるボタンが押されたら終了
65             if event.type == pygame.QUIT:
66                 pygame.quit() # の終了 Pygame画面閉じられる ()
67                 sys.exit() # スクリプト実行の終了
68             if event.type == pygame.KEYDOWN:
69                 if event.key == pygame.K_ESCAPE:
70                     pygame.quit() # の終了 Pygame画面閉じられる ()
71                     sys.exit() # スクリプト実行の終了

```

```

72         if event.key == pygame.K_r:
73             pygame.quit() # の終了 Pygame画面閉じられる ()
74             return # main()の終了 →main()の呼び出し元が無限ループなのでまた初めから実行
75
76
77 if __name__ == "__main__":
78     while True:
79         main()

```

次に魚を表現するクラスである Fish.py のコードを示す。コンストラクタは 23 行目からである。コンストラクタでは魚の初期位置の中心三次元座標、速度三次元ベクトル、色のタプル、魚の大きさ、骨の分割数、四元数で初期向きを受け取り、各種パラメータを初期化している。ここで受け取った三次元座標を四元数として保持する。速度ベクトルについても同様である。魚の向きは四元数で保持している。また、加速したときの最大速度、最低速度を初速度から計算して保持している。メンバ変数である decoreation は魚の体を構成する頂点群を繋げて描画する単位ごとにリストにして、リストのリストとして保持している。これは体の長さや骨の分割数に応じて計算されている。

39 行目の set_decoration メソッドでは骨の一本の線しかない魚に対して、体を構成する頂点群を計算して魚らしい外見にするメソッドである。

70 行目から魚を描画する display メソッドが定義されている。引数として描画先のバッファ、ビュー変換行列、透視投影変換行列、画面の座標に変換するスケール、原点座標を受け取る。描画にはまず魚のモデリング座標と四元数の姿勢から回転をし、四元数の 4 次元座標ベクトルから同次座標系の 4 次元座標ベクトルに変換し、PPM とビュー行列を前からかけて描画する座標に変換し、簡易 viewport clipping を行い、画面の座標に変換してから線画を描画している。その後、魚の上下をわかりやすくするとさかのようなものを表示するメソッド display_landmark、身体を装飾を描画する display_decoration メソッド、魚の現在位置がわかりやすくなるように床面に影を投影して描画する display_shadow メソッドを呼び出している。

155 行目から魚の状態を更新する update メソッドが定義されている。魚が前進していて何らかのキー入力がある場合、input メソッドでキー入力処理を行う。速度に回転を状態のぶんだけ加え、オイラー法で更新している。update2P メソッドについては入力処理の input メソッドの代わりに input2P メソッドを呼び出す以外は同じである。

184 行目の input メソッドでは 1P のキー入力処理を行っている。WASD キーで上下左右の回転を行い、スペースキーで加速する。input2P メソッドでは矢印キーで上下左右の回転を行い、右コントロールキーで加速する。なお、update と input はともに入力としてキー入力、キーで外から干渉できるときの速度、オイラー法における時間刻み幅を受け取る。

death_judge メソッドでは魚が入力のポジションという円柱の軸のてっぺんに点を取り、その xz 平面方向に円形を描き、軸からの距離を計算し、接触判定を行うメソッド。xz 平面における y 軸平行のチンアナゴの中心軸からの距離がチンアナゴの半径より小さい場合、接触したと判定し True を返す。

Listing 2 Fish.py

```

1 import numpy as np
2 import pygame
3 from quaternions import *
4 from object_module import solid2plane
5
6 # 魚クラス
7 class Fish:

```

```

8  # コンストラクタ
9  # 引数
10 # position魚の中心位置:
11 # velocity魚の速度ベクトル:
12 # color魚の色:(RGB)
13 # size魚の大きさ:
14 # segment魚の分割数:
15 # orientation魚の回転クォータニオン:
16 # メンバ変数
17 # position魚の中心位置次元ベクトル:(4)
18 # color魚の色:(RGB)
19 # size魚の大きさ:
20 # segment魚の骨の分割数:
21 # segments魚の各セグメント位置次元ベクトル:(4)リスト
22 # display_segments表示用魚の各セグメント位置次元ベクトル:(4)リスト
23 def __init__(self, position = None, velocity = None, color = (0, 0, 255), size = 0.1,
24             segment = 2, orientation = np.array([1., 0., 0., 0.])):
25     self.position = np.concatenate([[0.],position]) # 魚の中心位置
26     self.color = color # 魚の色 (RGB)
27     self.size = size
28     self.segment = segment
29     self.segments = [np.array([0,0,0,size*_segment-size/2]) for _ in range(segment)]
30     self.display_segments = [np.array([0,0,0,size*_segment-size/2]) for _ in range(
31         segment)]
32     self.quaternion = orientation
33     self.velocity = (self.segments[0]-self.segments[-1])/np.linalg.norm(self.segments
34         [0]-self.segments[-1])*np.linalg.norm(velocity) if np.linalg.norm(velocity) > 0
35     else np.array([0.,0.,0.,0.])
36     self.v_MAX = np.linalg.norm(self.velocity) * 5
37     self.v_MIN = np.linalg.norm(self.velocity) * 1
38     self.decorations = []
39     self.set_decoration()
40     self.display_mode = True
41
42 # さかなの形を装飾するメソッド
43 def set_decoration(self):
44     tmp = []
45     for _ in range(self.segment):
46         tmp.append(self.segments[_] - np.array([0.,0.,np.sin(_*np.pi/(self.segment-1))*
47             self.size*0.25, 0.]))
48     self.decorations.append(tmp)
49     tmp = []
50     for _ in range(self.segment):
51         tmp.append(self.segments[_] - np.array([0.,np.sin(_*np.pi/(self.segment-1))*
52             self.size*0.1,0., 0.]))
53     self.decorations.append(tmp)
54     tmp = []
55     for _ in range(self.segment):
56         tmp.append(self.segments[_] + np.array([0.,0.,np.sin(_*np.pi/(self.segment-1))*
57             self.size*0.25, 0.]))
58     self.decorations.append(tmp)
59     tmp = []
60     for _ in range(self.segment):
61         tmp.append(self.segments[_] + np.array([0.,np.sin(_*np.pi/(self.segment-1))*
62             self.size*0.1,0., 0.]))
63     self.decorations.append(tmp)
64     tmp = []

```

```

57     tmp.append(self.segments[-2])
58     tmp.append(self.segments[-1] + np.array([0.,0.,self.size*0.2, 0.]))
59     tmp.append(self.segments[-1] + np.array([0.,0.,-self.size*0.2, 0.]))
60     tmp.append(self.segments[-2])
61     self.decorations.append(tmp)
62
63     # 魚表示関数
64     # 引数
65     # buf描画先バッファ:
66     # Vビュー変換行列:
67     # PPM透視投影行列:
68     # scrcentr画面中心座標:
69     # scale画面サイズの変換拡大率:
70     def display(self, buf, V, PPM, scrcentr, scale):
71         if not self.display_mode:return
72         inv_q = inverse_quaternion(self.quaternion)
73         for i in range(self.segment-1):
74             p1 = mult_quaternion(mult_quaternion(self.quaternion, self.display_segments[i])
75                                   , inv_q)
76             p2 = mult_quaternion(mult_quaternion(self.quaternion, self.display_segments[i
77               +1]), inv_q)
78             p1 = PPM @ V @ np.concatenate([(p1+self.position)[1:4], [1.]])
79             p2 = PPM @ V @ np.concatenate([(p2+self.position)[1:4], [1.]])
80             line_points = solid2plane(p1, p2)
81             if line_points is not None:
82                 p1 = line_points[0][0:2]/line_points[0][3]
83                 p1 = scale * p1 + scrcentr
84                 p2 = line_points[1][0:2]/line_points[1][3]
85                 p2 = scale * p2 + scrcentr
86                 pygame.draw.line(buf, self.color, p1, p2, 2)
87                 self.display_landmark(buf, V, PPM, scrcentr, scale)
88                 self.display_decoration(buf, V, PPM, scrcentr, scale)
89                 self.display_shadow(buf, V, PPM, scrcentr, scale)
90
91     # 魚のデコレーション表示関数
92     # 引数
93     # buf描画先バッファ:
94     # Vビュー変換行列:
95     # PPM透視投影行列:
96     # scrcentr画面中心座標:
97     # scale画面サイズの変換拡大率:
98     def display_decoration(self, buf, V, PPM, scrcentr, scale):
99         if not self.display_mode:return
100         inv_q = inverse_quaternion(self.quaternion)
101         for decoration in self.decorations:
102             for i in range(len(decoration)-1):
103                 p1 = mult_quaternion(mult_quaternion(self.quaternion, decoration[i]), inv_q)
104                 p2 = mult_quaternion(mult_quaternion(self.quaternion, decoration[i+1]),
105                                       inv_q)
106                 p1 = PPM @ V @ np.concatenate([(p1+self.position)[1:4], [1.]])
107                 p2 = PPM @ V @ np.concatenate([(p2+self.position)[1:4], [1.]])
108                 line_points = solid2plane(p1, p2)
109                 if line_points is not None:
110                     p1 = line_points[0][0:2]/line_points[0][3]
111                     p1 = scale * p1 + scrcentr
112                     p2 = line_points[1][0:2]/line_points[1][3]
113                     p2 = scale * p2 + scrcentr

```

```

111         pygame.draw.line(buf, self.color, p1, p2, 1)
112     for i in range(len(self.decorations[0])):
113         for j in range(4):
114             jter = j%4
115             p1 = mult_quaternion(mult_quaternion(self.quaternion, self.decorations[jter
116                                     ][i]), inv_q)
117             p2 = mult_quaternion(mult_quaternion(self.quaternion, self.decorations[jter
118                                     ][i]), inv_q)
119             p1 = PPM @ V @ np.concatenate([(p1+self.position)[1:4], [1.]])
120             p2 = PPM @ V @ np.concatenate([(p2+self.position)[1:4], [1.]])
121             line_points = solid2plane(p1, p2)
122             if line_points is not None:
123                 p1 = line_points[0][0:2]/line_points[0][3]
124                 p1 = scale * p1 + screncentr
125                 p2 = line_points[1][0:2]/line_points[1][3]
126                 p2 = scale * p2 + screncentr
127                 pygame.draw.line(buf, self.color, p1, p2, 1)
128
129 # 魚の目印表示関数
130 # 引数
131 # buf描画先バッファ:
132 # Vビュー変換行列:
133 # PPM透視投影行列:
134 # screncentr画面中心座標:
135 # scale画面サイズの変換拡大率:
136 # color目印の色:(RGB)
137 def display_landmark(self,buf,V,PPM,screncentr,scale,color=(255,0,0)):
138     if not self.display_mode:return
139     inv_q = inverse_quaternion(self.quaternion)
140     p1 = mult_quaternion(mult_quaternion(self.quaternion, self.display_segments[0]),
141                           inv_q)
142     p2 = mult_quaternion(mult_quaternion(self.quaternion, self.display_segments[1]+np.
143                           array([0.,0.,self.size/5,0.])), inv_q)
144     p1 = PPM @ V @ np.concatenate([(p1+self.position)[1:4], [1.]])
145     p2 = PPM @ V @ np.concatenate([(p2+self.position)[1:4], [1.]])
146     line_points = solid2plane(p1, p2)
147     if line_points is not None:
148         p1 = line_points[0][0:2]/line_points[0][3]
149         p1 = scale * p1 + screncentr
150         p2 = line_points[1][0:2]/line_points[1][3]
151         p2 = scale * p2 + screncentr
152         pygame.draw.line(buf, color, p1, p2, 2)
153
154 # 魚の状態更新関数
155 # 引数
156 # key_inputキーボード入力情報:
157 # key_vel速度変化量:
158 def update(self,key_input=None, key_vel=0.0,dt=1.0):
159     if key_input is not None and key_vel > 0.0:
160         self.input(key_input, key_vel, dt)
161
162     v1 = mult_quaternion(mult_quaternion(self.quaternion, self.velocity),
163                           inverse_quaternion(self.quaternion))
164     self.position += v1*dt
165     if self.segment >= 2:
166         temp = np.linalg.norm(self.velocity)

```

```

163         self.velocity = self.segments[0]-self.segments[-1]
164         self.velocity = self.velocity/np.linalg.norm(self.velocity)*temp
165     # 魚の状態更新関数
166     # 引数
167     # key_inputキーボード入力情報:
168     # key_vel速度変化量:
169     def update2P(self, key_input=None, key_vel=0.0, dt=1.0):
170         if key_input is not None and key_vel > 0.0:
171             self.input2P(key_input, key_vel, dt)
172
173         v1 = mult_quaternion(mult_quaternion(self.quaternion, self.velocity),
174                             inverse_quaternion(self.quaternion))
175         self.position += v1*dt
176         if self.segment >= 2:
177             temp = np.linalg.norm(self.velocity)
178             self.velocity = self.segments[0]-self.segments[-1]
179             self.velocity = self.velocity/np.linalg.norm(self.velocity)*temp
180
181     # 魚の入力処理関数
182     # 引数
183     # key_inputキーボード入力情報:
184     # key_vel速度変化量:
185     def input(self, key_input, key_vel, dt):
186         leng = np.linalg.norm(self.segments[0]-self.segments[-1])
187         # 加速減速の制御
188         if key_input[pygame.K_SPACE]:
189             if np.linalg.norm(self.velocity) < self.v_MAX:
190                 self.velocity *= 1.05*dt
191             else:
192                 self.velocity = self.velocity/np.linalg.norm(self.velocity)*self.v_MAX
193         else:
194             if np.linalg.norm(self.velocity) > self.v_MIN:
195                 self.velocity *= 0.95*dt
196             else:
197                 self.velocity = self.velocity/np.linalg.norm(self.velocity)*self.v_MIN
198
199         if key_input[pygame.K_w]:
200             self.quaternion = mult_quaternion(self.quaternion, np.array([np.cos(key_vel/2),
201                                     np.sin(key_vel/2), 0., 0.]))
202             for _ in range(self.segment):
203                 self.display_segments[_] = self.segments[_] - np.array([0., 0., key_vel*leng*(
204                     np.sin(_*np.pi/self.segment) - 0.5*0), 0.])*10
205         if key_input[pygame.K_s]:
206             self.quaternion = mult_quaternion(self.quaternion, np.array([np.cos(-key_vel/2),
207                                     np.sin(-key_vel/2), 0., 0.]))
208             for _ in range(self.segment):
209                 self.display_segments[_] = self.segments[_] + np.array([0., 0., key_vel*leng*(
210                     np.sin(_*np.pi/self.segment) - 0.5*0), 0.])*10
211         if key_input[pygame.K_a]:
212             self.quaternion = mult_quaternion(self.quaternion, np.array([np.cos(-key_vel/2),
213                                     np.sin(-key_vel/2), 0., 0.]))
214             for _ in range(self.segment):
215                 self.display_segments[_] = self.segments[_] - np.array([0., key_vel*leng*(np.
216                     sin(_*np.pi/self.segment) - 0.5*0), 0., 0.])*10
217         if key_input[pygame.K_d]:
218             self.quaternion = mult_quaternion(self.quaternion, np.array([np.cos(key_vel/2),
219                                     np.sin(key_vel/2), 0., 0.]))

```



```

212         for _ in range(self.segment):
213             self.display_segments[_] = self.segments[_] + np.array([0.,key_vel*leng*(np.
                sin(_*np.pi/self.segment) - 0.5*0), 0.,0.])*10
214     if not key_input[pygame.K_w] and not key_input[pygame.K_s] and not key_input[pygame
        .K_a] and not key_input[pygame.K_d]:
215         for _ in range(self.segment):
216             self.display_segments[_] = self.segments[_]
217
218     # 魚の入力処理関数
219     # 引数
220     # key_inputキーボード入力情報:
221     # key_vel速度変化量:
222     def input2P(self, key_input, key_vel, dt):
223         leng = np.linalg.norm(self.segments[0]-self.segments[-1])
224         # 加速減速の制御
225         if key_input[pygame.K_RCTRL]:
226             if np.linalg.norm(self.velocity) < self.v_MAX:
227                 self.velocity *= 1.05*dt
228             else:
229                 self.velocity = self.velocity/np.linalg.norm(self.velocity)*self.v_MAX
230         else:
231             if np.linalg.norm(self.velocity) > self.v_MIN:
232                 self.velocity *= 0.95*dt
233             else:
234                 self.velocity = self.velocity/np.linalg.norm(self.velocity)*self.v_MIN
235
236         if key_input[pygame.K_UP]:
237             self.quaternion = mult_quaternion(self.quaternion, np.array([np.cos(key_vel/2),
                np.sin(key_vel/2), 0., 0.]))
238             for _ in range(self.segment):
239                 self.display_segments[_] = self.segments[_] - np.array([0.,0.,key_vel*leng*(
                    np.sin(_*np.pi/self.segment) - 0.5*0), 0.])*10
240         if key_input[pygame.K_DOWN]:
241             self.quaternion = mult_quaternion(self.quaternion, np.array([np.cos(-key_vel/2)
                , np.sin(-key_vel/2), 0., 0.]))
242             for _ in range(self.segment):
243                 self.display_segments[_] = self.segments[_] + np.array([0.,0.,key_vel*leng*(
                    np.sin(_*np.pi/self.segment) - 0.5*0), 0.])*10
244         if key_input[pygame.K_LEFT]:
245             self.quaternion = mult_quaternion(self.quaternion, np.array([np.cos(-key_vel/2)
                , 0., np.sin(-key_vel/2), 0.]))
246             for _ in range(self.segment):
247                 self.display_segments[_] = self.segments[_] - np.array([0.,key_vel*leng*(np.
                    sin(_*np.pi/self.segment) - 0.5*0), 0.,0.])*10
248         if key_input[pygame.K_RIGHT]:
249             self.quaternion = mult_quaternion(self.quaternion, np.array([np.cos(key_vel/2),
                0., np.sin(key_vel/2), 0.]))
250             for _ in range(self.segment):
251                 self.display_segments[_] = self.segments[_] + np.array([0.,key_vel*leng*(np.
                    sin(_*np.pi/self.segment) - 0.5*0), 0.,0.])*10
252         if not key_input[pygame.K_UP] and not key_input[pygame.K_DOWN] and not key_input[
            pygame.K_LEFT] and not key_input[pygame.K_RIGHT]:
253             for _ in range(self.segment):
254                 self.display_segments[_] = self.segments[_]
255
256     # 魚の情報表示関数
257

```

```

258     def print_info(self):
259         print("Position:\t", self.position)
260         print("Velocity:\t", self.velocity)
261         print("Color:\t", self.color)
262         print("Size:\t", self.size)
263         print("Segment:\t", self.segment)
264         print("Segments:\t", self.segments)
265
266     # 魚の死亡判定関数
267     # 引数
268     # position判定対象の位置次元ベクトル:(4)
269     # radius判定対象の半径:
270     def death_judge(self, position:np.array,radius:float):
271         if np.sqrt(np.pow(self.position[1]-position[1],2)+np.pow(self.position[3]-position
272             [3],2)) < radius and self.position[2] < position[2]:
273             return True
274         else:
275             return False
276
277     # 魚の影表示関数
278     # 引数
279     # buf描画先バッファ:
280     # Vビュー変換行列:
281     # PPM透視投影行列:
282     # scrcentr画面中心座標:
283     # scale画面サイズの変換拡大率:
284     def display_shadow(self, buf, V, PPM, scrcentr, scale):
285         if not self.display_mode:return
286         inv_q = inverse_quaternion(self.quaternion)
287         for i in range(self.segment-1):
288             p1 = mult_quaternion(mult_quaternion(self.quaternion, self.display_segments[i])
289                 , inv_q)
289             p2 = mult_quaternion(mult_quaternion(self.quaternion, self.display_segments[i
290                 +1]), inv_q)
291             p1 = p1+self.position
292             p2 = p2+self.position
293             p1[2] = 0.
294             p2[2] = 0.
295             p1 = PPM @ V @ np.concatenate([(p1)[1:4], [1.]])
296             p2 = PPM @ V @ np.concatenate([(p2)[1:4], [1.]])
297             line_points = solid2plane(p1, p2)
298             if line_points is not None:
299                 p1 = line_points[0][0:2]/line_points[0][3]
300                 p1 = scale * p1 + scrcentr
301                 p2 = line_points[1][0:2]/line_points[1][3]
302                 p2 = scale * p2 + scrcentr
303                 pygame.draw.line(buf, (255, 0, 0), p1, p2, 2)
304
305     # テスト用メイン実行部
306     if __name__ == "__main__":
307         fish = Fish()
308         fish.print_info()

```

次に、チンアナゴを表現するクラスである ChineseEel.py のコードを示す。

コンストラクタは7行目からである。コンストラクタではチンアナゴの初期位置の中心三次元座標、速度三

次元ベクトル、色のタプル、チンアナゴの大きさを受け取り、各種パラメータを初期化している。チンアナゴの向きは y 軸方向に固定されているため、四元数で保持していない。なお、大きさは半径と最長時の長さで指定できる。

15 行目から display メソッドが定義されている。引数として描画先のバッファ、ビュー変換行列、透視投影変換行列、画面の座標に変換するスケール、原点座標を受け取る。チンアナゴは地面から伸びてくるため、ちゃんと地表に出たもののみ描画したい。ポリゴンならば陰面消去や Z バッファで実現できるが、今回ポリゴンのように辺を決まった順で定義していないので、難しい。そこで、チンアナゴの先端の y 座標がどこにあるかを判定し、地面より下にある部分を計算的に描画しないようにした。その後、チンアナゴのモデリング座標を同次座標系の 4 次元座標ベクトルに変換し、PPM とビュー行列を前からかけて描画する座標に変換し、簡易 viewport clipping を行い、画面の座標に変換してから線画を描画している。

102 行目からチンアナゴの状態を更新する update メソッドが定義されている。地面から伸びてきて、指定した最大長さに達していない場合、y 座標方向に速度を加え、オイラー法で更新している。また、チンアナゴが伸び切った後は速度反転し、地面に戻るようになっている。なお、魚の死亡のために get_position メソッドで現在位置を取得できるようにしている。

Listing 3 ChineseEel.py

```
1 import numpy as np
2 import pygame
3 from quaternions import *
4 from object_module import solid2plane, get_circle
5 #チンアナゴクラス敵 ()
6 class ChineseEel:
7     def __init__(self, position = None, velocity = 0.004, radius = 0.01, length = 0.4,
8         color = (200, 200, 0)):
9         self.position = np.concatenate([[0.],[position[0]], [0.],[position[1]]])
10        self.color = color
11        self.radius = radius
12        self.length = length
13        self.velocity = velocity
14
15    def display(self, buf, V, PPM, scrcentr, scale):
16        if self.position[2]<0:
17            return
18        elif self.position[2] < self.radius*2:
19            tmp = [np.array([0.,self.position[2]*0.5,0.,0.]),np.array([0.,-self.position
20                [2]*0.5,0.,0.]),np.array([0.,0.,self.position[2]*0.5]),np.array
21                ([0.,0.,0.,-self.position[2]*0.5])]
22            for i in range(4):
23                p1 = self.position
24                p2 = self.position + tmp[i] - self.position[2]*np.array([0.,0.,1.,0.])
25                p1 = PPM @ V @ np.concatenate([p1[1:4], [1.]])
26                p2 = PPM @ V @ np.concatenate([p2[1:4], [1.]])
27                line_points = solid2plane(p1, p2)
28                if line_points is not None:
29                    p1 = line_points[0][0:2]/line_points[0][3]
30                    p1 = scale * p1 + scrcentr
31                    p2 = line_points[1][0:2]/line_points[1][3]
32                    p2 = scale * p2 + scrcentr
33                    pygame.draw.line(buf, self.color, p1, p2, 2)
34            circle_points = get_circle(self.radius, 20)
```

```

33     for i in range(len(circle_points)):
34         p1 = self.position + circle_points[i] - self.position[2]*np.array
           ([0.,0.,1.,0.])
35         p2 = self.position + circle_points[(i+1)%len(circle_points)] - self.position
           [2]*np.array([0.,0.,1.,0.])
36         p1 = PPM @ V @ np.concatenate([p1[1:4], [1.]])
37         p2 = PPM @ V @ np.concatenate([p2[1:4], [1.]])
38         line_points = solid2plane(p1, p2)
39         if line_points is not None:
40             p1 = line_points[0][0:2]/line_points[0][3]
41             p1 = scale * p1 + scrcentr
42             p2 = line_points[1][0:2]/line_points[1][3]
43             p2 = scale * p2 + scrcentr
44             pygame.draw.line(buf, self.color, p1, p2, 2)
45     else:
46         tmp = [np.array([0.,self.radius,0.,0.]),np.array([0.,-self.radius,0.,0.]),np.
           array([0.,0.,0.,self.radius]),np.array([0.,0.,0.,-self.radius])]
47         for i in range(4):
48             p1 = self.position
49             p2 = self.position + tmp[i] - 2*self.radius*np.array([0.,0.,1.,0.])
50             p3 = p2.copy()
51             p3[2] = 0.
52             p1 = PPM @ V @ np.concatenate([p1[1:4], [1.]])
53             p2 = PPM @ V @ np.concatenate([p2[1:4], [1.]])
54             p4 = p2.copy()
55             p3 = PPM @ V @ np.concatenate([p3[1:4], [1.]])
56             line_points = solid2plane(p1, p2)
57             if line_points is not None:
58                 p1 = line_points[0][0:2]/line_points[0][3]
59                 p1 = scale * p1 + scrcentr
60                 p2 = line_points[1][0:2]/line_points[1][3]
61                 p2 = scale * p2 + scrcentr
62                 pygame.draw.line(buf, self.color, p1, p2, 2)
63             line_points = solid2plane(p3, p4)
64             if line_points is not None:
65                 p3 = line_points[0][0:2]/line_points[0][3]
66                 p3 = scale * p3 + scrcentr
67                 p4 = line_points[1][0:2]/line_points[1][3]
68                 p4 = scale * p4 + scrcentr
69                 pygame.draw.line(buf, self.color, p3, p4, 2)
70         circle_points = get_circle(self.radius, 20)
71         circle_points = circle_points + self.position - 2*self.radius*np.array
           ([0.,0.,1.,0.])
72         while circle_points[0][2] > 0:
73             for i in range(len(circle_points)):
74                 p1 = circle_points[i]
75                 p2 = circle_points[(i+1)%len(circle_points)]
76                 p1 = PPM @ V @ np.concatenate([p1[1:4], [1.]])
77                 p2 = PPM @ V @ np.concatenate([p2[1:4], [1.]])
78                 line_points = solid2plane(p1, p2)
79                 if line_points is not None:
80                     p1 = line_points[0][0:2]/line_points[0][3]
81                     p1 = scale * p1 + scrcentr
82                     p2 = line_points[1][0:2]/line_points[1][3]
83                     p2 = scale * p2 + scrcentr
84                     pygame.draw.line(buf, self.color, p1, p2, 2)
85             circle_points = circle_points - np.array([0.,0.,0.02,0.])

```

```

86
87     for i in range(len(circle_points)):
88         circle_points[i][2] = max(circle_points[i][2], 0.)
89     for i in range(len(circle_points)):
90         p1 = circle_points[i]
91         p2 = circle_points[(i+1)%len(circle_points)]
92         p1 = PPM @ V @ np.concatenate([p1[1:4], [1.]])
93         p2 = PPM @ V @ np.concatenate([p2[1:4], [1.]])
94         line_points = solid2plane(p1, p2)
95         if line_points is not None:
96             p1 = line_points[0][0:2]/line_points[0][3]
97             p1 = scale * p1 + scrcentr
98             p2 = line_points[1][0:2]/line_points[1][3]
99             p2 = scale * p2 + scrcentr
100         pygame.draw.line(buf, self.color, p1, p2, 2)
101
102     def update(self, dt):
103         self.position[2] += self.velocity*dt
104         if self.position[2] > self.length:
105             self.velocity *= -1
106         if self.position[2] < -1e-3:
107             return True
108         return False
109
110     def get_position(self):
111         return self.position
112
113     def get_radius(self):
114         return self.radius

```

次に Frame クラスを示す。水槽の枠を表現するクラスである。コンストラクタは 8 行目からである。コンストラクタでは枠の中心三次元座標、フレームの姿勢を表す四元数、垂直方向つまり奥行き方向の分割数、水平つまり横方向の分割数、垂直方向の全体の大きさ、水平方向の全体の大きさを受け取り、各種パラメータを初期化している。フレームの姿勢を表す四元数は今回初期値での変更以外で変えようがなく、斜面でもできるようにしたかったため設定した。垂直方向つまり奥行き方向の分割数、水平つまり横方向の分割数、垂直方向の全体の大きさ、水平方向の全体の大きさは保持せず、直接描画する辺の頂点群のモデリング座標に変換してリストのリストとして保持している。

37 行目から display メソッドが定義されている。引数として描画先のバッファ、ビュー変換行列、透視投影変換行列、画面の座標に変換するスケール、原点座標、色のタプルを受け取る。他のオブジェクトと違い色は引数で受け取るようにしている。

Listing 4 Frame.py

```

1 import numpy as np
2 import pygame
3 from quaternions import *
4 from object_module import solid2plane
5
6 # 格子フレームクラス
7 # ステージの枠組みを表示するためのクラス
8 class Frame:
9     # コンストラクタ
10    # 引数
11    # position フレームの中心位置:

```

```

12 # orientation フレームの回転クォータニオン:
13 # vertical_segment 垂直方向の分割数:
14 # horizontal_segment 水平方向の分割数:
15 # vertical_size 垂直方向の全体大きさ:
16 # horizontal_size 水平方向の全体大きさ:
17 def __init__(self, position = np.zeros(3), orientation = np.array([1., 0., 0., 0.]),
    vertical_segment = 10, horizontal_segment = 10, vertical_size = 1, horizontal_size
    = 1):
18     self.position = np.concatenate([[0.], position])
19     self.orientation = orientation
20     self.vertical_edges = [[np.array([0., vertical_size*(_/vertical_segment-0.5), 0.,
    horizontal_size*0.5]), np.array([0., vertical_size*(_/vertical_segment-0.5), 0.,
    -horizontal_size*0.5])] for _ in range(vertical_segment+1)]
21     self.horizontal_edges = [[np.array([0., vertical_size*0.5, 0., horizontal_size*(_/
    horizontal_segment-0.5)]), np.array([0., -vertical_size*0.5, 0., horizontal_size
    *(_/horizontal_segment -0.5)])] for _ in range(horizontal_segment+1)]
22
23 # フレーム位置設定関数
24 # 引数
25 # position フレームの中心位置:
26 def set_position(self, position: np.array):
27     self.position = np.concatenate([[0.], position])
28
29 # フレーム表示関数
30 # 引数
31 # buf 描画先バッファ:
32 # V ビュー変換行列:
33 # PPM 透視投影行列:
34 # scrcentr 画面中心座標:
35 # scale 画面サイズへの変換拡大率:
36 # color フレームの色: (RGB)
37 def display(self, buf, V, PPM, scrcentr, scale, color = (255, 255, 255)):
38     for edge in self.vertical_edges:
39         p1 = mult_quaternion(mult_quaternion(self.orientation, edge[0]),
    inverse_quaternion(self.orientation)) + self.position
40         p2 = mult_quaternion(mult_quaternion(self.orientation, edge[1]),
    inverse_quaternion(self.orientation)) + self.position
41         p1 = PPM @ V @ np.concatenate([p1[1:4], [1.]])
42         p2 = PPM @ V @ np.concatenate([p2[1:4], [1.]])
43         line_points = solid2plane(p1, p2)
44         if line_points is not None:
45             p1 = line_points[0][0:2]/line_points[0][3]
46             p1 = scale * p1 + scrcentr
47             p2 = line_points[1][0:2]/line_points[1][3]
48             p2 = scale * p2 + scrcentr
49             pygame.draw.line(buf, color, p1, p2, 2)
50     for edge in self.horizontal_edges:
51         p1 = mult_quaternion(mult_quaternion(self.orientation, edge[0]),
    inverse_quaternion(self.orientation)) + self.position
52         p2 = mult_quaternion(mult_quaternion(self.orientation, edge[1]),
    inverse_quaternion(self.orientation)) + self.position
53         p1 = PPM @ V @ np.concatenate([p1[1:4], [1.]])
54         p2 = PPM @ V @ np.concatenate([p2[1:4], [1.]])
55         line_points = solid2plane(p1, p2)
56         if line_points is not None:
57             p1 = line_points[0][0:2]/line_points[0][3]
58             p1 = scale * p1 + scrcentr

```

```

59         p2 = line_points[1][0:2]/line_points[1][3]
60         p2 = scale * p2 + scrcentr
61         pygame.draw.line(buf, color, p1, p2, 2)
62     tmp_edges = [[self.horizontal_edges[0][0],self.horizontal_edges[0][0]+np.array
        ([0.,0.,1.,0.])],
63                 [self.horizontal_edges[0][1],self.horizontal_edges[0][1]+np.array
        ([0.,0.,1.,0.])],
64                 [self.horizontal_edges[-1][0],self.horizontal_edges[-1][0]+np.array
        ([0.,0.,1.,0.])],
65                 [self.horizontal_edges[-1][1],self.horizontal_edges[-1][1]+np.array
        ([0.,0.,1.,0.])]]
66     for edge in tmp_edges:
67         p1 = mult_quaternion(mult_quaternion(self.orientation, edge[0]),
        inverse_quaternion(self.orientation)) + self.position
68         p2 = mult_quaternion(mult_quaternion(self.orientation, edge[1]),
        inverse_quaternion(self.orientation)) + self.position
69         p1 = PPM @ V @ np.concatenate([p1[1:4], [1.]])
70         p2 = PPM @ V @ np.concatenate([p2[1:4], [1.]])
71         line_points = solid2plane(p1, p2)
72         if line_points is not None:
73             p1 = line_points[0][0:2]/line_points[0][3]
74             p1 = scale * p1 + scrcentr
75             p2 = line_points[1][0:2]/line_points[1][3]
76             p2 = scale * p2 + scrcentr
77             pygame.draw.line(buf, color, p1, p2, 2)

```

Camera クラスを示す。カメラを表現するクラスである。コンストラクタは8行目からである。現在位置、注視点、視界の bottom,left,top,right,near の7つのパラメータを受け取り、各種パラメータを初期化している。このパラメータから透視投影変換行列を計算し、保持している。また、ビュー変換行列も計算し、保持している。

input メソッドなどが定義されている。本来視点を動かせるようにしていたが、インタラクションを考えて全体カメラ一つに固定した。よって、実際今回は変換行列を計算して保持、出力するだけのクラスになっている。

Listing 5 Camera.py

```

1 import numpy as np
2 from quaternions import *
3 import pygame
4
5
6 class Camera:
7     # コンストラクタ
8     def __init__(self, position=np.array([0.,0.,0.]), lookat=np.array([0.,0.,1.]),top=1,
        bottom=-1, left=-1, right=1, near=1.0):
9         self.position = position
10        self.lookat = lookat
11        self.orientation = np.array([1., 0., 0., 0.])
12        self.top = top
13        self.bottom = bottom
14        self.left = left
15        self.right = right
16        self.near = near
17        self.setPPMatrix()
18        self.setViewMatrix()

```

```

19
20
21
22 # perspective projection matrix 設定
23 def setPPMatrix(self):
24     self.PPM = np.array([[2*self.near/(self.left-self.right), 0, (self.right + self.
25         left)/(self.left-self.right), 0],
26         [0, 2*self.near/(self.top - self.bottom), (self.top + self.bottom
27             )/(self.top - self.bottom), 0],
28         [0, 0, - 1, -2*self.near],
29         [0, 0, -1, 0]])
30
31 # view matrix 設定
32 def setViewMatrix(self,upper=np.array([0,1,0])):
33     f = (self.lookat - self.position)/np.linalg.norm(self.lookat - self.position)
34     s = np.cross(f, upper)/np.linalg.norm(np.cross(f, upper))
35     u = np.cross(s, f)
36     # カメラ座標系変換行列の作成
37     self.VM = np.array([[s[0], s[1], s[2], -np.dot(s, self.position)],
38         [u[0], u[1], u[2], -np.dot(u, self.position)],
39         [-f[0], -f[1], -f[2], np.dot(f, self.position)],
40         [0, 0, 0, 1]])
41
42 # 系メソッド get
43 def ViewMatrix(self):
44     return self.VM
45 def PPMatrix(self):
46     return self.PPM
47
48 # クォータニオン回転
49 # 引数
50 # axis回転軸ベクトル:
51 # angle回転角度ラジアン:()
52 def quaternion_rotate(self, axis, angle):
53     q = np.concatenate([[np.cos(angle/2)], axis/np.linalg.norm(axis)*np.sin(angle/2)])
54     q_inv = inverse_quaternion(q)
55     # p = np.concatenate([[0.], self.position])
56     p = np.concatenate([[0.], self.lookat])
57     p_rot = mult_quaternion(mult_quaternion(q, p), q_inv)
58     self.lookat = p_rot[1:4]
59     # self.position = p_rot[1:4]
60
61
62 def FPS(self, obj_position, target_position):
63     self.position = obj_position
64     self.lookat = target_position
65
66 # キー入力処理
67 # 引数
68 # keysキー入力情報:
69 # Key_velキー入力による移動量:
70 # quate_controlクォータニオン回転ロドリゲス回転切り替え:/
71 def input(self,keys, Key_vel,quate_control:bool=True):
72
73     # u d l r translate
74     if not keys[pygame.K_LCTRL]:
75         A = np.zeros(3)
76         if keys[pygame.K_a]:
77             A[1] += 1

```



```

74         if keys[pygame.K_d]:
75             A[1] += -1
76         if keys[pygame.K_w]:
77             A[0] += 1
78         if keys[pygame.K_s]:
79             A[0] += -1
80         # front back translate
81         if keys[pygame.K_q]:
82             A[2] += 1
83         if keys[pygame.K_e]:
84             A[2] += -1
85
86         if np.linalg.norm(A) > 0:
87             # self.position -= self.lookat
88             if quate_control:
89                 self.quaternion_rotate(A, Key_vel)
90             else:
91                 RotationM = np.array([(1-np.cos(Key_vel))*A[0]**2+np.cos(Key_vel), (1-
92                     np.cos(Key_vel))*A[0]*A[1]-A[2]*np.sin(Key_vel), (1-np.cos(Key_vel))
93                     *A[0]*A[2]+A[1]*np.sin(Key_vel)],
94                     [(1-np.cos(Key_vel))*A[0]*A[1]+A[2]*np.sin(Key_vel),
95                     (1-np.cos(Key_vel))*A[1]**2+np.cos(Key_vel), (1-np
96                     .cos(Key_vel))*A[1]*A[2]-A[0]*np.sin(Key_vel)],
97                     [(1-np.cos(Key_vel))*A[2]*A[0]-A[1]*np.sin(Key_vel),
98                     (1-np.cos(Key_vel))*A[1]*A[2]+A[0]*np.sin(Key_vel)
99                     , (1-np.cos(Key_vel))*A[2]**2+np.cos(Key_vel)]
100
101                 ])
102                 self.position = RotationM @ self.position
103             # self.position += self.lookat
104         else:
105             # u d l r translate
106             if keys[pygame.K_a]:
107                 self.position[0] -= Key_vel
108                 self.lookat[0] -= Key_vel
109             if keys[pygame.K_d]:
110                 self.position[0] += Key_vel
111                 self.lookat[0] += Key_vel
112             if keys[pygame.K_w]:
113                 self.position[1] += Key_vel
114                 self.lookat[1] += Key_vel
115             if keys[pygame.K_s]:
116                 self.position[1] -= Key_vel
117                 self.lookat[1] -= Key_vel
118
119             # front back translate
120             if keys[pygame.K_q]:
121                 self.position[2] += Key_vel
122                 self.lookat[2] += Key_vel
123             if keys[pygame.K_e]:
124                 self.position[2] -= Key_vel
125                 self.lookat[2] -= Key_vel
126
127     def print_info(self):
128         print("Position:\t", self.position)
129         print("Lookat:\t\t", self.lookat)
130         # print("Orientation:\t", self.orientation)
131         # print("PPM:\t", self.PPM)

```

StringsBlock クラスを示す。文字列表示を行うクラスである。コンストラクタは 13 行目からである。文字列、フォントサイズ、二次元座標、色のタプルを受け取り、各種パラメータを初期化している。コンストラクタではそれ以降の A から Z までの文字の頂点を計算するメソッドから頂点群を計算し、リストを作成して保持している。

28 行目から display メソッドが定義されている。引数として描画先のバッファ、画面の座標に変換するスケール、原点座標を受け取る。リストの要素である文字単位の頂点群を順に取り出し、print_string メソッドで画面に描画している。

36 行目から print_string メソッドが定義されている。引数として描画先のバッファ、画面の座標に変換するスケール、原点座標、文字単位の頂点群を受け取る。各頂点群を画面の座標に変換し、線画を描画している。

Listing 6 StringsBlock.py

```
1 import numpy as np
2 import pygame
3
4
5 # 文字列描画クラス
6 class StringsBlock:
7     # コンストラクタ
8     # 引数
9     # strings 表示する文字列:
10    # font_size 文字サイズ:
11    # position 文字描画開始位置左上: ()
12    # color 文字色: (RGB)
13    def __init__(self, strings:str = "",font_size:float=0.075,position:np.array=np.array
14        ([-0.995,-0.92]),color:tuple=(255,255,0)):
15        self.str_strings = strings
16        self.font_size = font_size
17        self.position = position
18        self.strings = []
19        self.color = color
20        for i in range(len(strings)):
21            char_func = getattr(self, strings[i].upper(), None)
22            if callable(char_func):
23                self.strings.append(char_func(index=i))
24
25    # 文字列描画関数
26    # 引数
27    # buf 描画先バッファ:
28    # scrcentr 画面中心座標:
29    # scale 画面サイズへの変換拡大率:
30    def display(self, buf, scrcentr=np.array([0,0]), scale=1):
31        for char in self.strings:
32            self.print_char(buf, char=char, scrcentr=scrcentr, scale=scale)
33
34    # 2文字描画関数 D
35    # 引数
36    # buf 描画先バッファ:
37    # font_size 文字サイズ:
38    # point 文字描画開始位置左上: ()
39    def print_char(self, buf, char=np.array(None), scrcentr=np.array([0,0]), scale=0):
40        for i in range(len(char)-1):
41            pygame.draw.line(buf, self.color, scale * char[i] + scrcentr, scale * char[i+1]
42                + scrcentr, 1) # 直線の描
43            画
44
45    # A-の文字定義関数 Z
```

```

41 # 引数
42 # index文字列中の文字位置オリジン:(0)
43 def A(self,index=0)->np.array:
44     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,-self.
        font_size],[self.font_size/2,0],[self.font_size,-self.font_size],[self.
        font_size*3/4,-self.font_size/2],[self.font_size/4,-self.font_size/2]])
45 def B(self,index=0)->np.array:
46     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,-self.
        font_size],[0,0],[self.font_size*2/3,0],[self.font_size*5/6,-self.font_size
        /6],[self.font_size*5/6,-self.font_size/3],[self.font_size*2/3,-self.font_size
        /2],[0,-self.font_size/2],[self.font_size*5/6,-self.font_size/2],[self.
        font_size,-self.font_size*2/3],[self.font_size,-self.font_size*5/6],[self.
        font_size*5/6,-self.font_size],[0,-self.font_size]])
47 def C(self,index=0)->np.array:
48     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[self.
        font_size,-self.font_size/3],[self.font_size*2/3,0],[self.font_size/3,0],[0,-
        self.font_size/3],[0,-self.font_size*2/3],[self.font_size/3,-self.font_size],[
        self.font_size*2/3,-self.font_size],[self.font_size,-self.font_size*2/3]])
49 def D(self,index=0)->np.array:
50     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,-self.
        font_size],[0,0],[self.font_size*2/3,0],[self.font_size,-self.font_size/3],[
        self.font_size,-self.font_size*2/3],[self.font_size*2/3,-self.font_size],[0,-
        self.font_size]])
51 def E(self,index=0)->np.array:
52     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[self.
        font_size,-self.font_size],[0,-self.font_size],[0,-self.font_size/2],[self.
        font_size,-self.font_size/2],[0,-self.font_size/2],[0,0],[self.font_size,0]])
53 def F(self,index=0)->np.array:
54     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,-self.
        font_size],[0,-self.font_size/2],[self.font_size*3/4,-self.font_size/2],[0,-
        self.font_size/2],[0,0],[self.font_size,0]])
55 def G(self,index=0)->np.array:
56     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[self.
        font_size,-self.font_size/3],[self.font_size*2/3,0],[self.font_size/3,0],[0,-
        self.font_size/3],[0,-self.font_size*2/3],[self.font_size/3,-self.font_size],[
        self.font_size*2/3,-self.font_size],[self.font_size,-self.font_size*2/3],[self.
        font_size,-self.font_size/2],[self.font_size/2,-self.font_size/2]])
57 def H(self,index=0)->np.array:
58     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,0],[0,-
        self.font_size],[0,-self.font_size/2],[self.font_size,-self.font_size/2],[self.
        font_size,0],[self.font_size,-self.font_size]])
59 def I(self,index=0)->np.array:
60     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,0],[self.
        font_size,0],[self.font_size/2,0],[self.font_size/2,-self.font_size],[0,-self.
        font_size],[self.font_size,-self.font_size]])
61 def J(self,index=0)->np.array:
62     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[self.
        font_size,0],[self.font_size,-self.font_size*2/3],[self.font_size*2/3,-self.
        font_size],[self.font_size/3,-self.font_size],[0,-self.font_size*2/3]])
63 def K(self,index=0)->np.array:
64     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,0],[0,-
        self.font_size],[0,-self.font_size/2],[self.font_size,0],[0,-self.font_size
        /2],[self.font_size,-self.font_size]])
65 def L(self,index=0)->np.array:
66     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,0],[0,-
        self.font_size],[self.font_size,-self.font_size]])
67 def M(self,index=0)->np.array:

```

```

68         return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,-self.
        font_size],[0,0],[self.font_size/2,-self.font_size*3/4],[self.font_size,0],[
        self.font_size,-self.font_size]])
69 def N(self,index=0)->np.array:
70     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,-self.
        font_size],[0,0],[self.font_size,-self.font_size],[self.font_size,0]])
71 def O(self,index=0)->np.array:
72     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[self.
        font_size,-self.font_size/3],[self.font_size*2/3,0],[self.font_size/3,0],[0,-
        self.font_size/3],[0,-self.font_size*2/3],[self.font_size/3,-self.font_size],[
        self.font_size*2/3,-self.font_size],[self.font_size,-self.font_size*2/3],[self.
        font_size,-self.font_size/3]])
73 def P(self,index=0)->np.array:
74     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,-self.
        font_size],[0,0],[self.font_size,0],[self.font_size,-self.font_size/2],[0,-self.
        font_size/2]])
75 def Q(self,index=0)->np.array:
76     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[self.
        font_size,-self.font_size/3],[self.font_size*2/3,0],[self.font_size/3,0],[0,-
        self.font_size/3],[0,-self.font_size*2/3],[self.font_size/3,-self.font_size],[
        self.font_size*2/3,-self.font_size],[self.font_size,-self.font_size*2/3],[self.
        font_size,-self.font_size/3],[self.font_size/2,-self.font_size/2],[self.
        font_size,-self.font_size]])
77 def R(self,index=0)->np.array:
78     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,-self.
        font_size],[0,0],[self.font_size,0],[self.font_size,-self.font_size/2],[0,-self.
        font_size/2],[self.font_size,-self.font_size*3/4],[self.font_size,-self.
        font_size]])
79 def S(self,index=0)->np.array:
80     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[self.
        font_size,-self.font_size/6],[self.font_size*5/6,0],[self.font_size/6,0],[0,-
        self.font_size/6],[0,-self.font_size/3],[self.font_size,-self.font_size*2/3],[
        self.font_size,-self.font_size*5/6],[self.font_size*5/6,-self.font_size],[self.
        font_size/6,-self.font_size],[0,-self.font_size*5/6]])
81 def T(self,index=0)->np.array:
82     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,0],[self.
        font_size,0],[self.font_size/2,0],[self.font_size/2,-self.font_size]])
83 def U(self,index=0)->np.array:
84     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,0],[0,-
        self.font_size*3/4],[self.font_size/3,-self.font_size],[self.font_size*2/3,-
        self.font_size],[self.font_size,-self.font_size*3/4],[self.font_size,0]])
85 def V(self,index=0)->np.array:
86     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,0],[self.
        font_size/2,-self.font_size],[self.font_size,0]])
87 def W(self,index=0)->np.array:
88     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,0],[self.
        font_size/4,-self.font_size],[self.font_size/2,-self.font_size/2],[self.
        font_size*3/4,-self.font_size],[self.font_size,0]])
89 def X(self,index=0)->np.array:
90     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,0],[self.
        font_size,-self.font_size],[self.font_size/2,-self.font_size/2],[self.font_size
        ,0],[0,-self.font_size]])
91 def Y(self,index=0)->np.array:
92     return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,0],[self.
        font_size/2,-self.font_size/2],[self.font_size,0],[self.font_size/2,-self.
        font_size/2],[self.font_size/2,-self.font_size]])
93 def Z(self,index=0)->np.array:

```

```

94         return self.position+np.array([self.font_size*1.2*index,0])+np.array([[0,0],[self.
            font_size,0],[0,-self.font_size],[self.font_size,-self.font_size]])

```

次に GameManager クラスを示す。ゲーム全体を管理するクラスである。12 行目からコンストラクタが定義されている。コンストラクタでは魚 2 匹、チンアナゴ複数、水槽の枠、カメラ、文字列表示オブジェクトを生成し、時間刻み幅 dt など各種パラメータを初期化している。

34 行目から update メソッドが定義されている。引数としてキー入力、時間刻み幅を受け取る。このメソッドで各オブジェクトの状態更新、描画、判定を行っていて、いわばゲームループの本体である。まず、魚 2 匹、チンアナゴ複数、水槽の枠、カメラの状態を update している。魚を操作しているときに難しくなっているので、キー入力があるときチンアナゴが増えるように、カウントダウンを行い、一定時間ごとにチンアナゴを追加している。

チンアナゴと魚からの接触判定を行い、接触した魚を死亡扱いにしている。最後に文字列表示オブジェクトの内容を更新している。

基本的にゲームに追加機能を実装するときはそれぞれオブジェクトを作ったのち、このクラスの update メソッドに追加していくことになる。

Listing 7 GameManager.py

```

1 import sys
2 import pygame
3 import numpy as np
4 from Camera import *
5 from Frame import *
6 from Fish import *
7 from StringsBlock import *
8 from ChineseEel import *
9 import random, time
10
11 class GameManager:
12     def __init__(self):
13         self.dt = 1.0
14         self.size = 1.0
15         self.camera = Camera(position=np.array([0.,0.4,0.]), lookat=np.array([0.,0.,1.]))
16         self.frame = Frame(position=np.array([0.,0.,1.]), vertical_segment=30,
17             horizontal_segment=30, vertical_size=self.size, horizontal_size=self.size)
18         self.fish = Fish(position=np.array([0.5,0.3,1.]), velocity=np.array([0.001,0.,0.]),
19             segment=10,color=(255,255,0),orientation=deg2quat(90, np.array([0.,1.,0.])))
20         self.fish2 = Fish(position=np.array([0.,0.3,1.]), velocity=np.array([0.001,0.,0.]),
21             segment=10,color=(0,255,0),orientation=deg2quat(90, np.array([0.,1.,0.])))
22
23         self.eels = []
24         self.strings = []
25         self.iter_count:int = 100
26         self.flag = False
27
28     def add_eel(self, position, velocity=0.004, radius=0.01, length=0.4, color=(0,0,255)):
29         eel = ChineseEel(position=position, velocity=velocity, radius=radius, length=length,
30             color=color)
31         self.eels.append(eel)
32
33     def add_string(self, string, position, font_size=0.05, color=(255,255,255)):

```

```

31         string_block = StringsBlock(strings=string, position=position, font_size=font_size,
32                                     color=color)
33         self.strings.append(string_block)
34
35     def update(self, key_input, key_vel, buf, scrcentr, scale):
36         self.camera.setViewMatrix()
37         VM = self.camera.ViewMatrix()
38         PPM = self.camera.PPMMatrix()
39
40         if key_input is not None:
41             self.iter_count -= 1
42             random.seed(time.time())
43             if self.iter_count < 50 :
44                 self.iter_count = 50
45             if random.randint(0, self.iter_count) <= 1:
46                 self.add_eel(position = np.array([random.uniform(-self.size/2+self.frame.
47                                                     position[1], self.size/2+self.frame.position[1]),random.uniform(-self.
48                                                     size/2+self.frame.position[3], self.size/2+self.frame.position[3])]),
49                             velocity = random.uniform(0.002, 0.01), radius = random.uniform(0.008,
50                                                     0.015)*10*0.5, length = random.uniform(0.2, 0.4))
51         else:
52             self.iter_count = 200
53
54         key = pygame.key.get_pressed()
55         self.fish.update(key_input=key, key_vel=key_vel, dt=self.dt)
56         self.fish2.update2P(key_input=key, key_vel=key_vel, dt=self.dt)
57         for eel in self.eels:
58             f = eel.update(dt=self.dt)
59             if f:
60                 del eel
61
62         for eel in self.eels:
63             self.flag = self.flag or self.fish.death_judge(eel.get_position(), eel.
64                                                         get_radius())
65             self.flag = self.flag or self.fish2.death_judge(eel.get_position(), eel.
66                                                         get_radius())
67         if self.flag:
68             self.springs = []
69             self.strings.append(StringsBlock(strings="Game_Over", position=np.array
70                                             ([-0.4,0.05]), font_size=0.1, color=(0,255,0)))
71             self.strings.append(StringsBlock(strings="Game_Over", position=np.array
72                                             ([-0.4+0.01,0.05-0.01/2]), font_size=0.1, color=(0,255,0)))
73             self.strings.append(StringsBlock(strings="Game_Over", position=np.array
74                                             ([-0.4+0.02,0.05-0.02/2]), font_size=0.1, color=(0,255,0)))
75         else:
76             self.frame.display(buf, VM, PPM, scrcentr, scale)
77             self.fish.display(buf, VM, PPM, scrcentr, scale)
78             self.fish2.display(buf, VM, PPM, scrcentr, scale)
79             for eel in self.eels:
80                 eel.display(buf, VM, PPM, scrcentr, scale)
81         for string in self.strings:
82             string.display(buf, scrcentr, scale)
83
84         # camera.input(pygame.key.get_pressed(), Key_vel, True)

```

最後に、補助関数群を示す。quaternions.py、object_module.py である。それぞれ、簡潔な説明をコメントアウトとしてつけている。まず、quaternions.py のコードを示す。

Listing 8 quaternions.py

```

1 import numpy as np
2
3 # クォータニオン演算関連関数
4
5 # クォータニオンの加算正直足すだけだからほとんど意味ない (
6 # 引数
7 # q1, q2クォータニオン: (np.array([w, x, y, z]))
8 # 戻り値
9 # クォータニオンの加算結果 (np.array([w, x, y, z]))
10 def add_quaternion(q1:np.array, q2:np.array) -> np.array:
11     return q1+q2
12 # クォータニオンの乗算
13 # 引数
14 # q1, q2クォータニオン: (np.array([w, x, y, z]))
15 # 戻り値
16 # クォータニオンの乗算結果 (np.array([w, x, y, z]))
17 def mult_quaternion(q1:np.array, q2:np.array) -> np.array:
18     out = np.zeros(4)
19     out[0] = q1[0]*q2[0] - q1[1]*q2[1] - q1[2]*q2[2] - q1[3]*q2[3]
20     out[1] = q1[0]*q2[1] + q1[1]*q2[0] + q1[2]*q2[3] - q1[3]*q2[2]
21     out[2] = q1[0]*q2[2] - q1[1]*q2[3] + q1[2]*q2[0] + q1[3]*q2[1]
22     out[3] = q1[0]*q2[3] + q1[1]*q2[2] - q1[2]*q2[1] + q1[3]*q2[0]
23     return out
24
25 # クォータニオンの共役
26 # 引数
27 # qクォータニオン: (np.array([w, x, y, z]))
28 # 戻り値
29 # クォータニオンの共役 (np.array([w, x, y, z]))
30 def conjugate_quaternion(q:np.array)->np.array:
31     return np.array([q[0], -q[1], -q[2], -q[3]])
32
33 # クォータニオンの逆元
34 # 引数
35 # qクォータニオン: (np.array([w, x, y, z]))
36 # 戻り値
37 # クォータニオンの逆元 (np.array([w, x, y, z]))
38 def inverse_quaternion(q:np.array)->np.array:
39     return conjugate_quaternion(q) / np.dot(q, q)
40
41 # 角度と回転軸からクォータニオンを生成
42 # 引数
43 # angle回転角度度: (
44 # axis回転軸ベクトル:
45 # 戻り値
46 # クォータニオン (np.array([w, x, y, z]))
47 def deg2quat(angle:float, axis:np.array = np.array([1., 0., 0.]))->np.array:
48     if np.linalg.norm(axis) <= 0:
49         return np.array([1., 0., 0., 0.])
50     rad = angle * np.pi / 180
51     axis = axis / np.linalg.norm(axis)
52     return np.array([np.cos(rad/2), axis[0]*np.sin(rad/2), axis[1]*np.sin(rad/2), axis[2]*

```

```
np.sin(rad/2)])
```

次に object_module.py のコードを示す。

Listing 9 object_module.py

```
1 import numpy as np
2 import pygame
3
4 # 直線が画面に映るかどうかを判定し、映る場合はその 2 座標を返す関数 D
5 # 引数
6 # p1 物体の始点の同次座標: (np.array([x,y,z,w]))
7 # p2 物体の終点の同次座標: (np.array([x,y,z,w]))
8 # 戻り値
9 # 映る場合: np.array([[x1,y1,w1],[x2,y2,w2]]) (x/w,y/w が画面上の座標 w)
10 # 映らない場合: None
11 def solid2plane(p1:np.array, p2:np.array)->np.array:
12     if p1[3] > 1e-3 and p2[3] > 1e-3:
13         return np.array([p1, p2])
14     elif p1[3] > 1e-3 and p2[3] <= 1e-3:
15         t = (1e-3 - p1[3])/(p2[3]-p1[3])
16         inter = p1 + t*(p2 - p1)
17         return np.array([p1, inter])
18     elif p1[3] <= 1e-3 and p2[3] > 1e-3:
19         t = (1e-3 - p1[3])/(p2[3]-p1[3])
20         inter = p1 + t*(p2 - p1)
21         return np.array([inter, p2])
22     else:
23         return None
24
25 # x-平面上の円を取得する関数 z
26 # 引数
27 # radius 円の半径:
28 # segment 円の分割数:
29 # 戻り値
30 # 円周上の点のリスト [np.array([0.,x1,0.,z1]), np.array([0.,x2,0.,z2]), ...]
31 def get_circle(radius, segment):
32     points = []
33     for i in range(segment):
34         theta = 2.0 * np.pi * i / segment
35         x = radius * np.cos(theta)
36         y = radius * np.sin(theta)
37         points.append(np.array([0., x, 0., y]))
38     return points
```

以上で、コードの説明を終わる。